Hybrid Argo Workflows with Kubernetes and Slurm

Panagiotis Georgakopoulos

Technical University of Crete

January 22, 2026

Thesis Committee

Supervisor:	Prof. Vasilis Samoladas
Committee:	Prof. Nikos Giatrakos
	Prof. Deligiannakis Antonios

Table of contents

Introduction

Objective

System Architecture

Experiments

Results

Motivation

- The convergence of cloud-native (Kubernetes) and high-performance computing (HPC) has become increasingly important for modern scientific and data driven workloads
- Modern pipelines often combine interactive or service-oriented data pre-processing stages with computationally intensive batch execution
- Slurm remains the dominant workload manager for HPC environments due to it's superior scheduling capabilities, fair-share and accounting mechanisms as well as strong performance characteristics

Hybrid Workloads and Applications

1. Distributed Machine Learning
2. Bio-informatics Workflows
3. Large-scale simulations (Material Science/Fluid Dynamics)
4. Big Data Analytics
5. Life Sciences
6. Climate Simulations/Weather Forecasting
7. Etc..

Main Concerns

- Existing integration approaches often require strong assumptions about the capabilities of the HPC environment, such as the availability of a REST API (slurmrestd daemon)
- Data transfer handling when collocation of data is absent
- The semantics of data movement
- Artifact visualization and monitoring across two distinct control planes
- Storage Constraints on the HPC Cluster
- Security Considerations

Objective

Objective

The core objective of this Master's Thesis is to introduce a novel Hybrid Workflow Orchestration tool that addresses these limitations and provides a user friendly solution with Quality of Life upgrades and extensibility options.

Approach

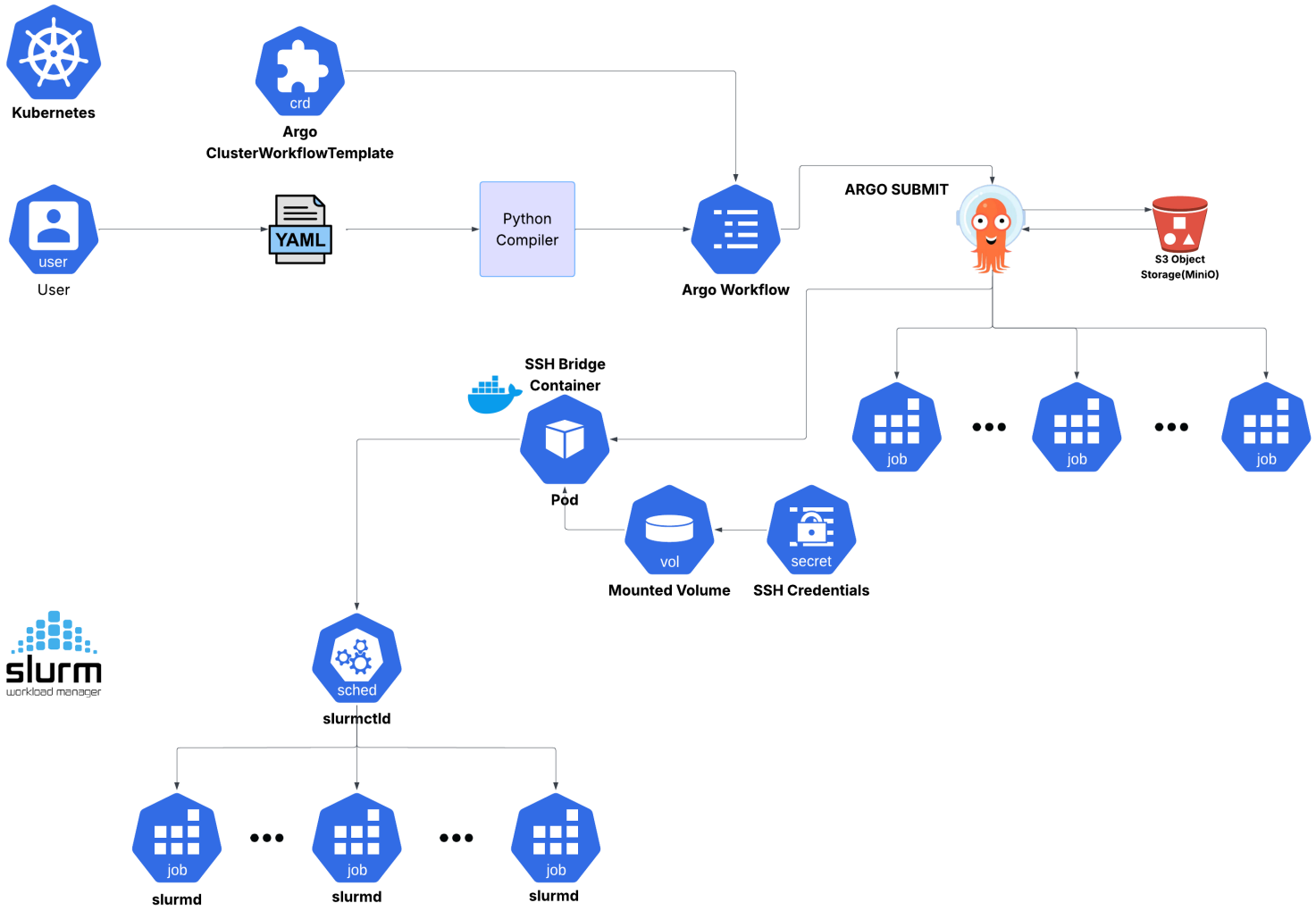
Our system follows the Adjacent model where the two control planes remain separate with the main focus being workflow-level orchestration through Argo Workflows on the Kubernetes plane.

Our design surfaces Slurm jobs as first-class workflow steps in Argo, preserving their distinct semantics and providing abstraction with a high level domain specific workflow description language as well as flexibility by allowing the user to apply custom scheduling constraints.

Contributions

- An abstract hybrid workflow description language
- A data handling layer
- Implied dependencies between steps derived from inputs and outputs
- Intermediate Step data clean-up mechanism
- A unified user-interface for logging, monitoring and artifact visualization through the ARGO UI
- Custom workflow-level Scheduling Constraints

System Overview



System Components

- A high-level YAML DSL for Hybrid Workflows.
- An automated compilation pipeline to Argo Workflows.
- Slurm execution via a reusable SSH-based Argo ClusterWorkflowTemplate.
- SSH Client Docker Image encapsulating submission logic and data handling
- SSH Credentials are stored as Kubernetes Secrets and passed to the Bridging container as Mounted Volumes.
- A data bridging mechanism using S3 artifact storage.
- Optional custom scheduler API for workflow-level policies.

Slurm Job Workflow Template Parameters

command	The command to be executed on Slurm
s3artifact	The input S3 Artifact Key
slurmInput	True if the input comes from a Slurm step
fetchData	True if the output is consumed by a K8s step
cleanData	True when CleanDataPath is set
cleanDataPath	The path for the clean-up
inputFilePath	File path for input (Optional)
outputFileName	The output file's path

Compilation

The python compiler processes the input DSL YAML file and produces an Argo Workflow where K8s type jobs are translated to inline DAG tasks while Slurm type jobs use the Slurm-job Workflow Template. The compiler initializes these parameters according to the data-flow relationships between steps and produces the DAG dependencies for each step. These parameters are passed to the SSH Bridge Container as Environment Variables while the Kubernetes Secrets containing the SSH Credentials are mounted as volumes.

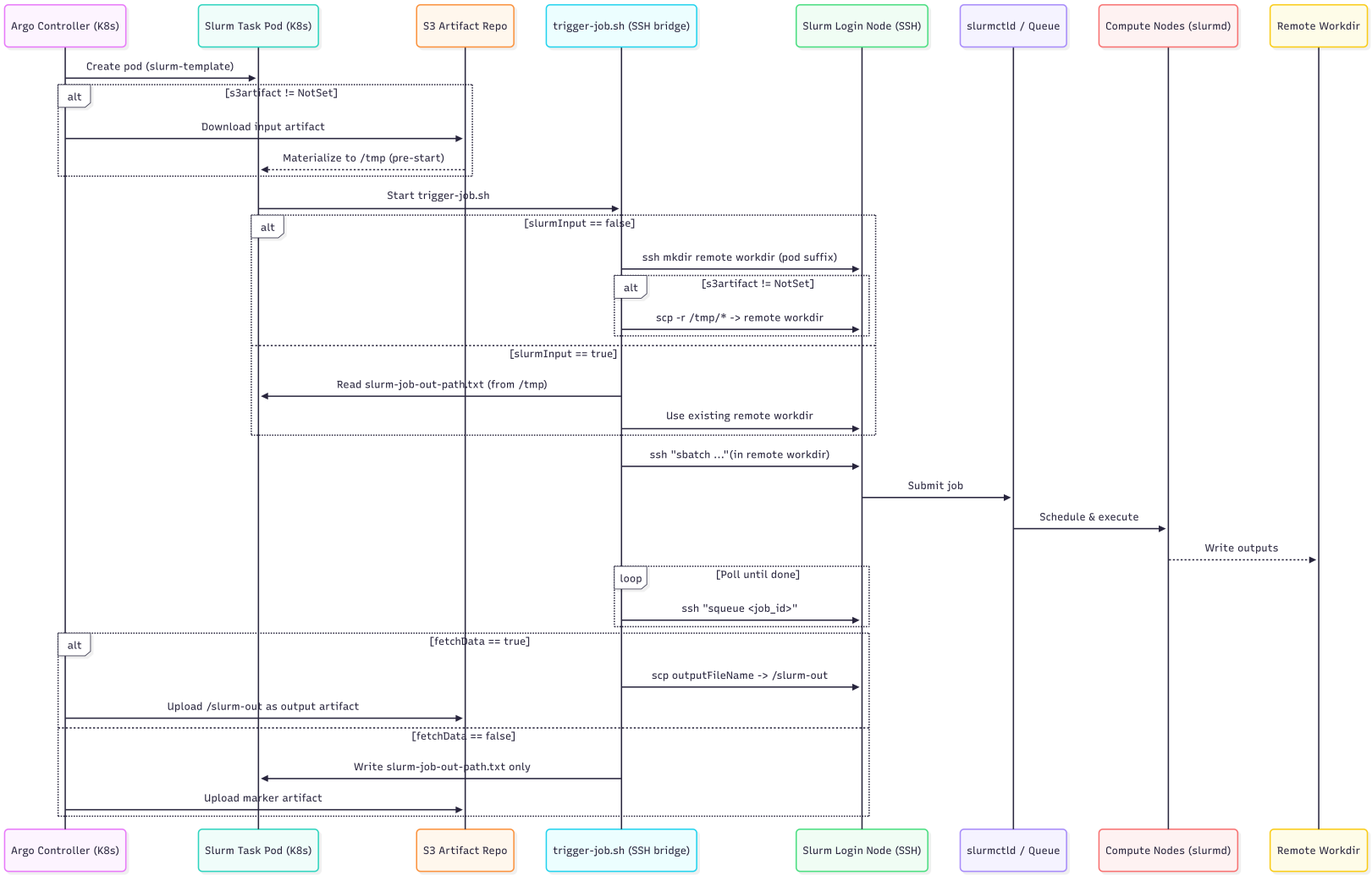
SSH Bridge Container

The SSH Bridge Container encapsulates the submission logic and handles the data exchange acting based on the Environment Variables set by the compiler. It is also responsible for the following sequence of actions in order to run the user's Workload on Slurm.

Execution Flow Steps

- Preparing Remote Execution Context.
- Transferring Input Artifacts.
- Reusing Context for Chained Slurm Jobs.
- Submitting the Slurm Job.
- Monitoring the Slurm Job.
- Retrieving Output Artifacts.
- Intermediate Slurm Step Data clean-up.

Execution Flow Sequence Diagram



Scheduler API

api.workflow	The full (mutable) Argo Workflow dictionary.
api.jobs	The original job list from the input DSL
api.dag_template	The compiled DAG template object that contains the list of DAG tasks.
api.tasks	A flat list of DAG task dictionaries (task["name"], task["dependencies"], task["arguments"], ...).
api.slurm_tasks	Subset of api.tasks by job type.
api.k8s_tasks	Subset of api.tasks by job type.
api.deps	A map name → dependencies computed from the DAG; useful for graph algorithms.
api.rdeps	Reverse dependencies name → dependents.

Helper Functions

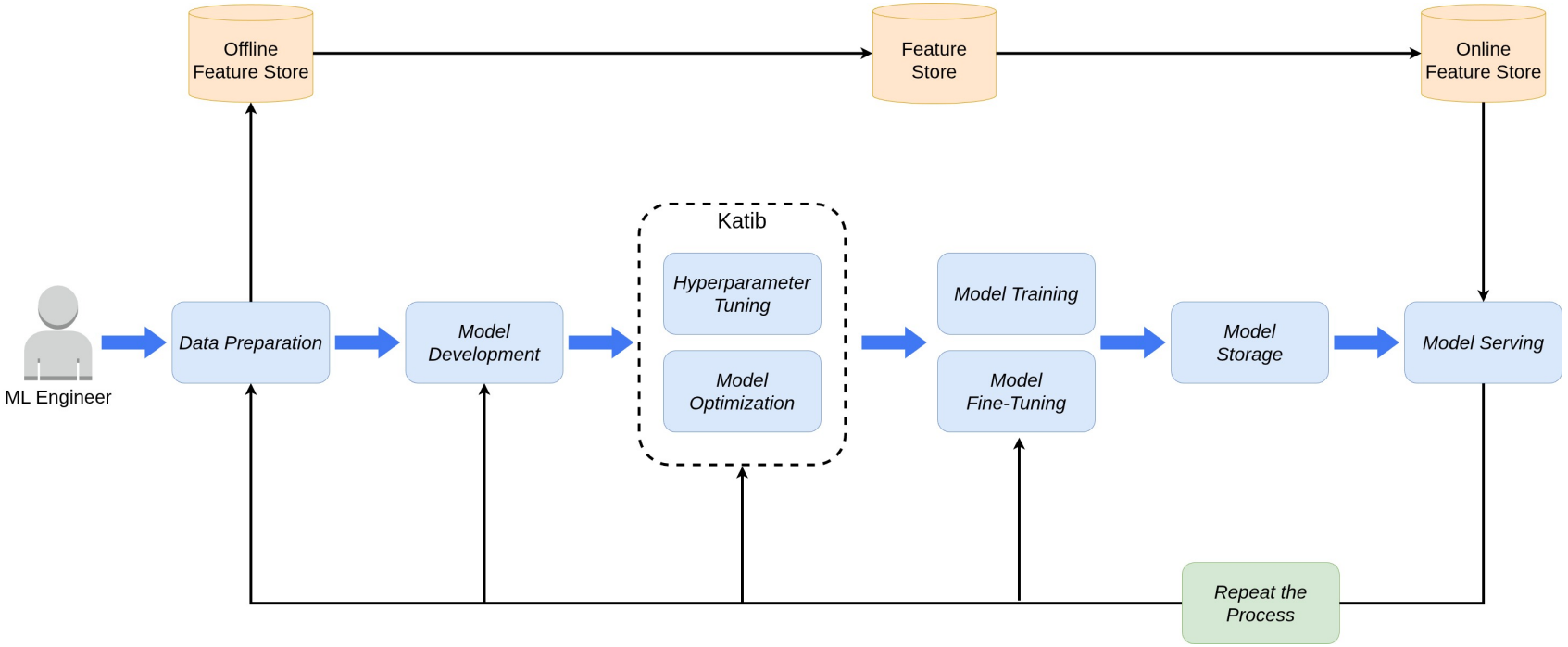
set_global_parallelism(n)	Sets a workflow-wide concurrency cap.
get_task(name)	Retrieves a DAG task by its unique name.
add_dependency(task, depends_on)	Ensures that one task starts only after another has finished.
chain([t_1, ..., t_k])	Serializes a list of tasks into a single lane. Adds edges $t_i \to t_{i+1}$ for all $i \in \{1, \ldots, k-1\}$.
select(pred)	Filters tasks based on an arbitrary predicate.

Experiments

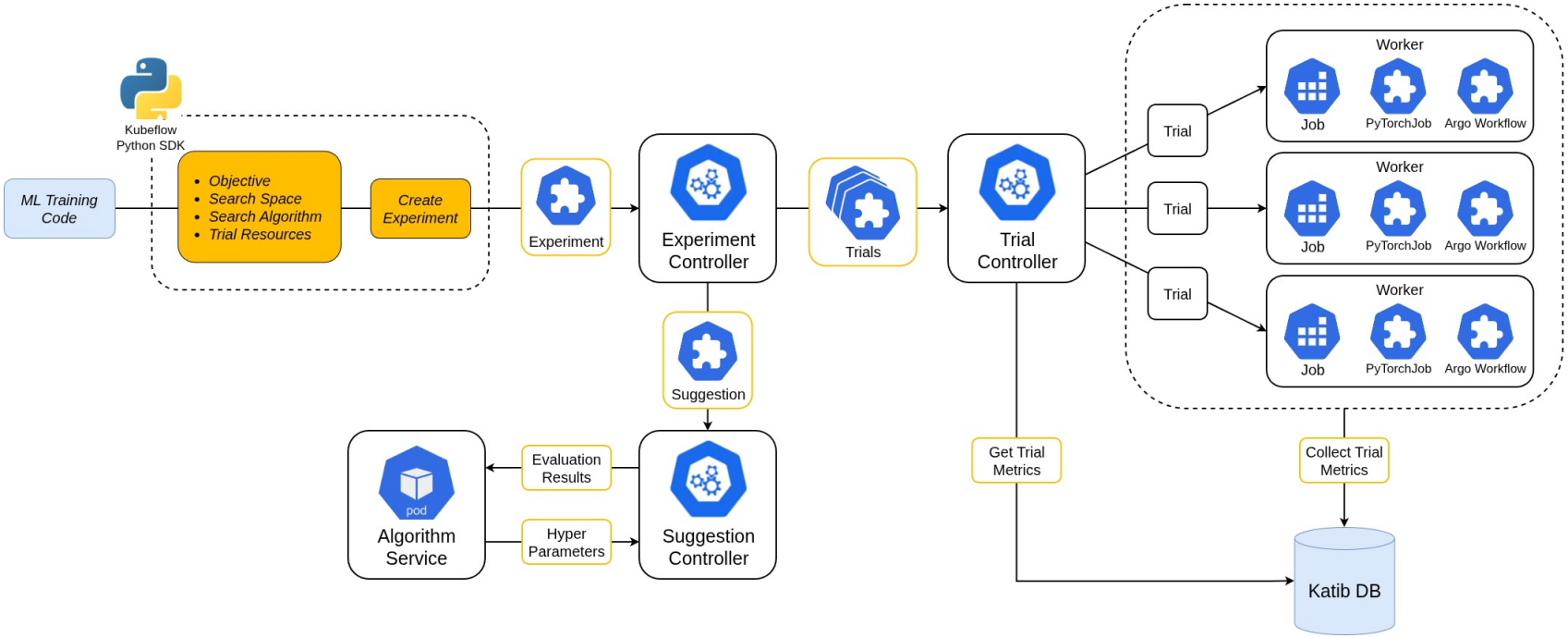
Evaluation on representative hybrid use cases.

1. Integration of Kubeflow Katib hyperparameter tuning with Slurm-backed trials
2. A Hybrid Genomic Quality Control Alignment Workflow that leverages Kubernetes for data preparation and Slurm for alignment.
3. A mock scenario demonstrating the practical utility of custom scheduling policies.

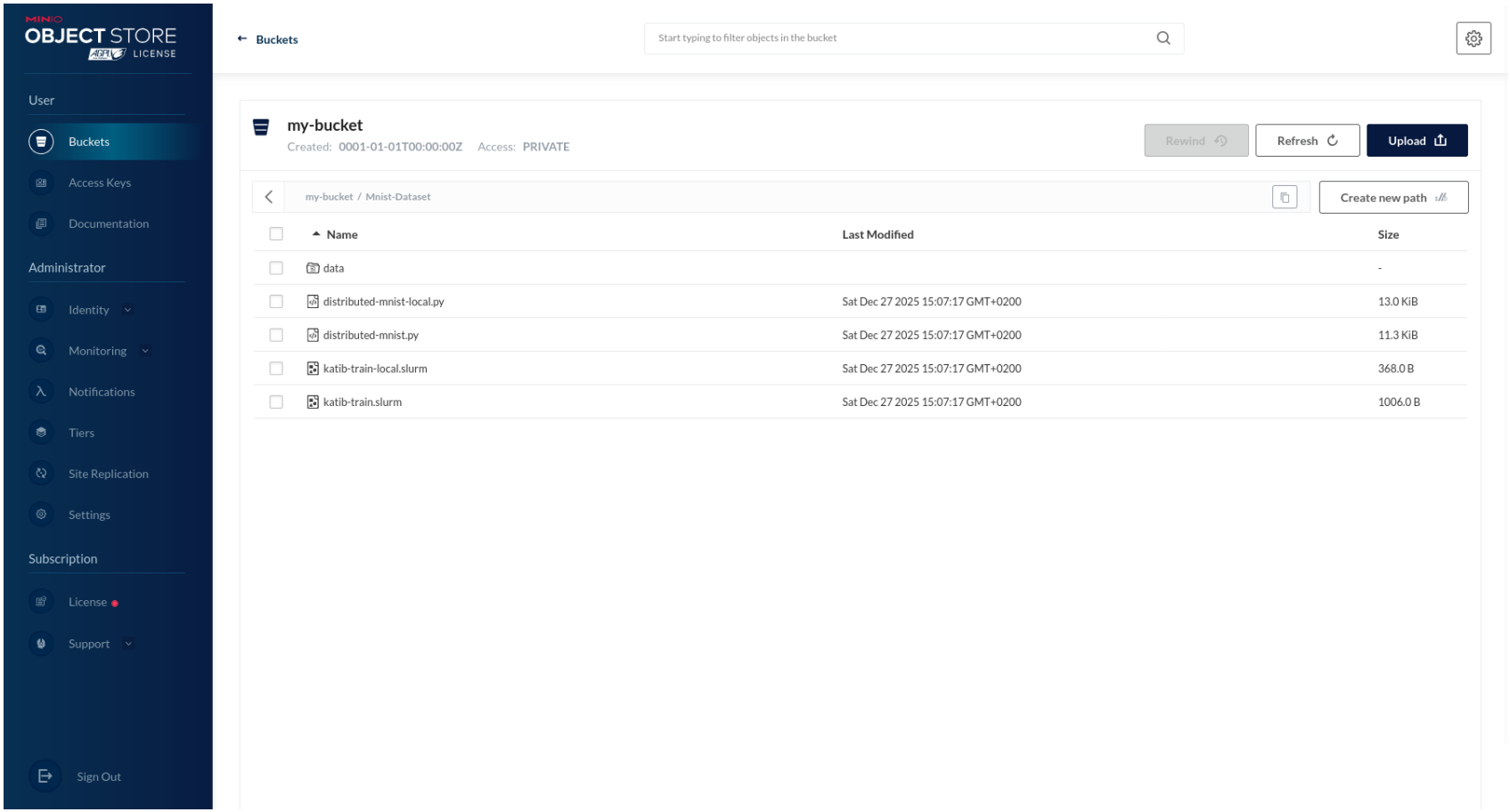
ML Lifecycle



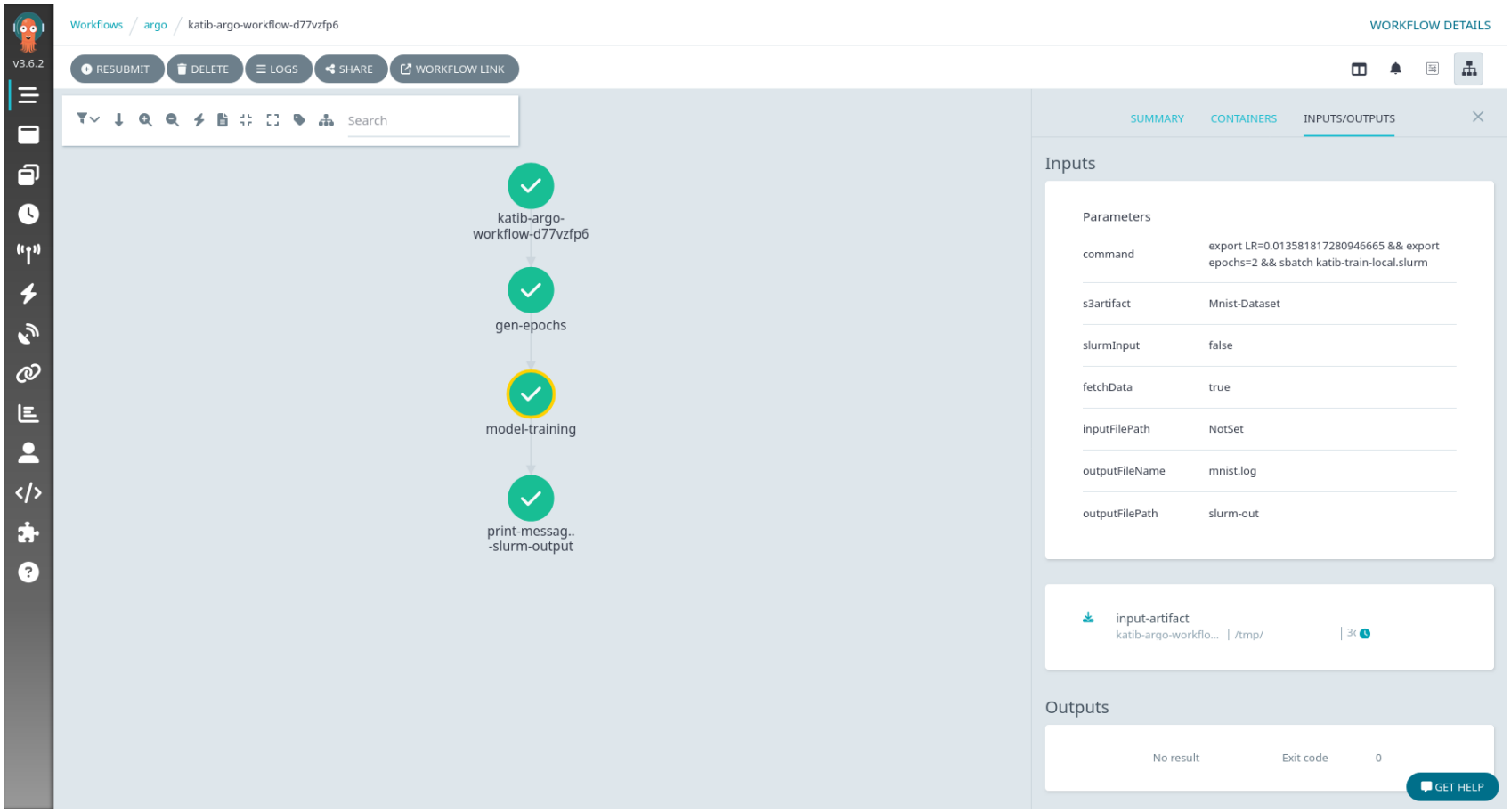
Katib Architecture



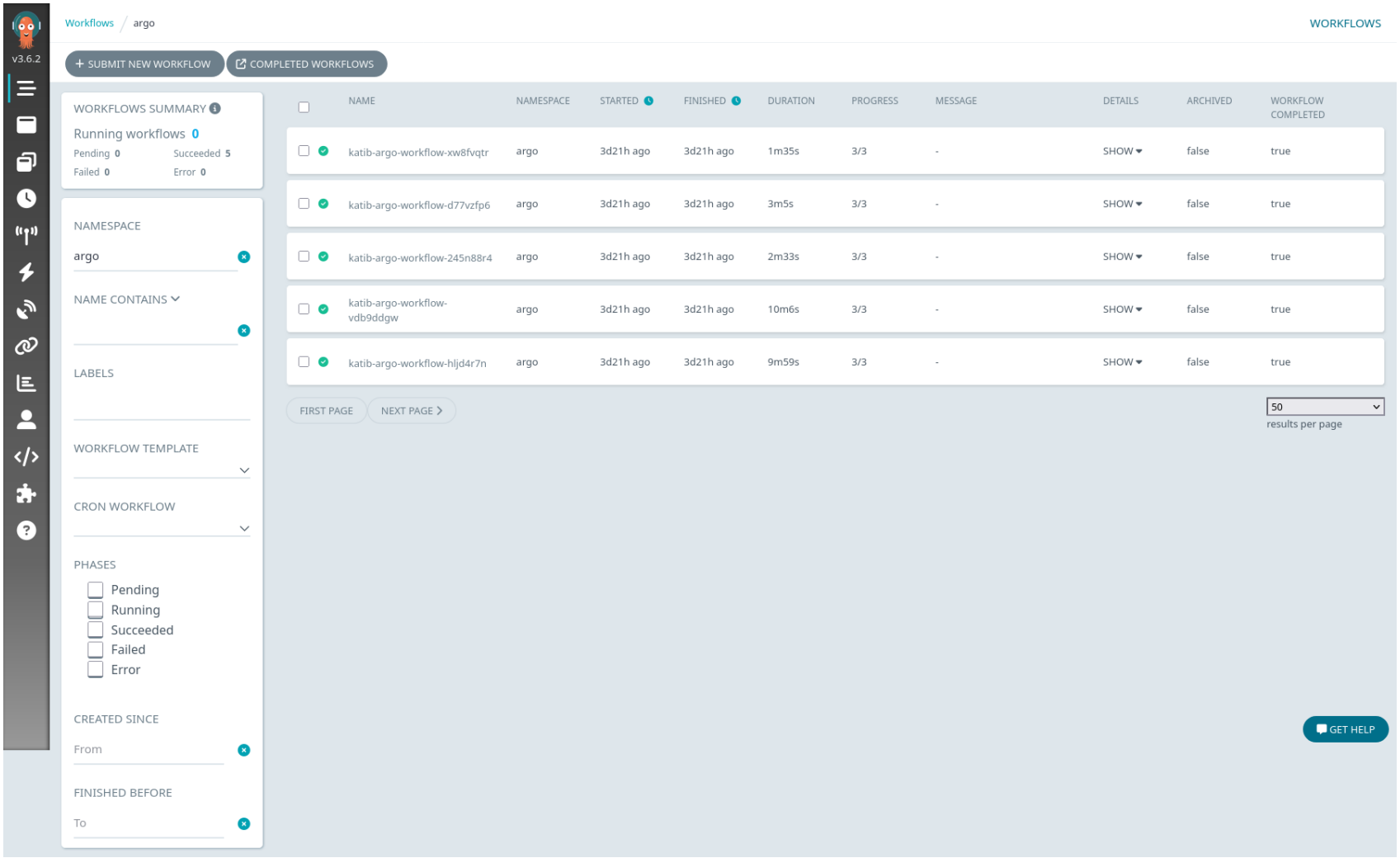
Input Staging on Minio (S3 Artifact Storage)



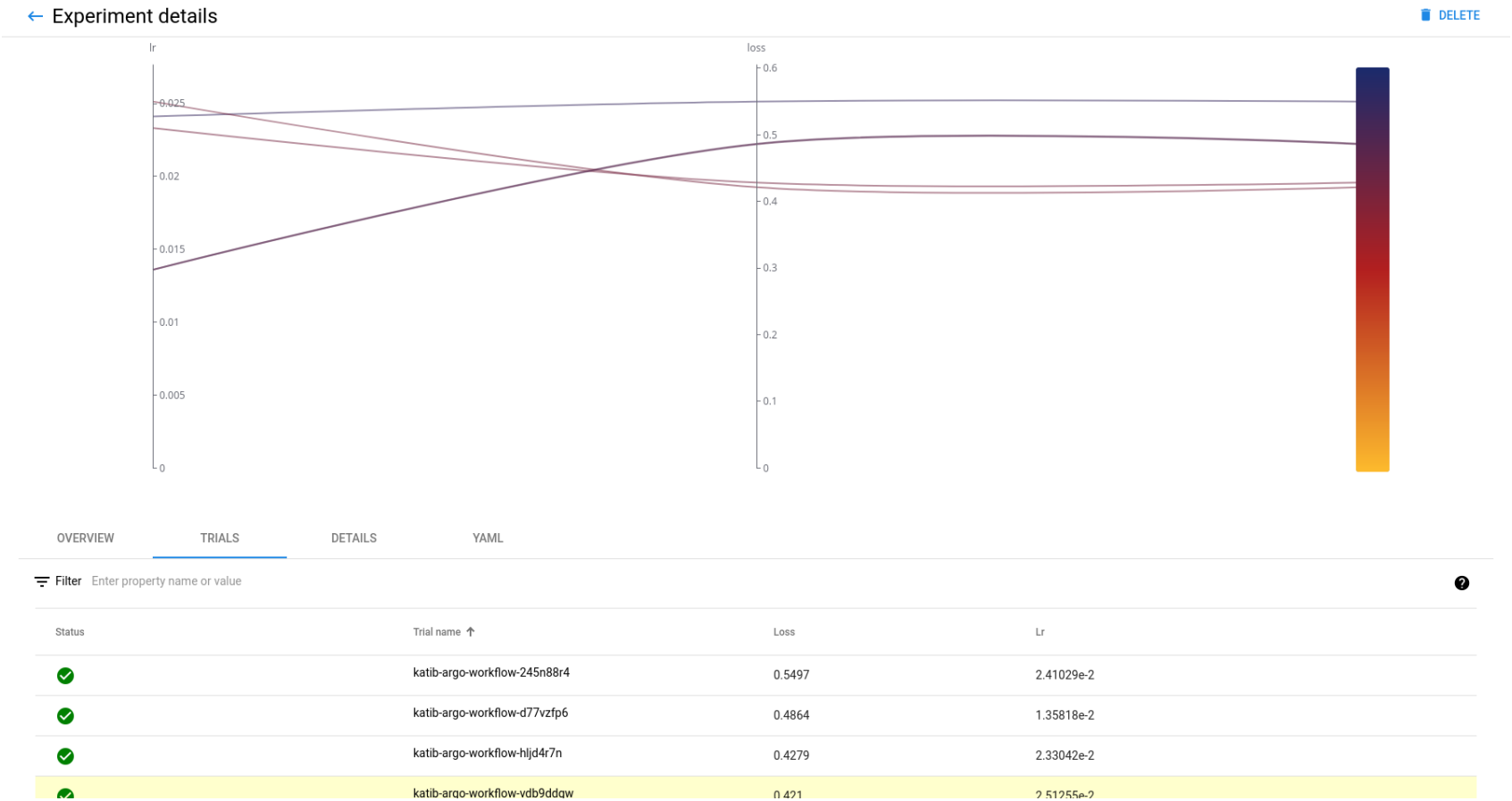
Katib Trial Workflow



Each Katib Trial is a separate Argo Workflow



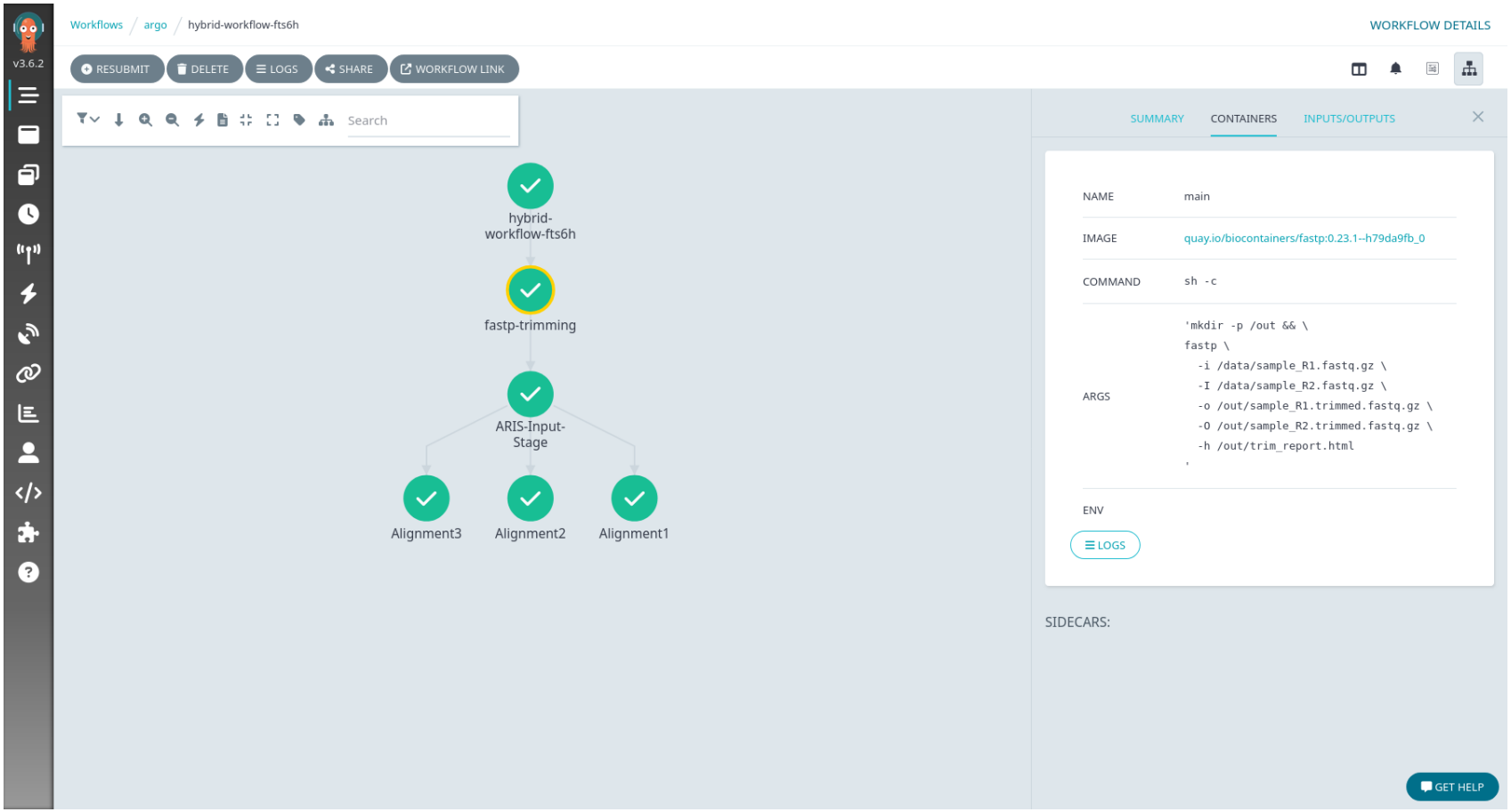
Katib UI


Experiment details
DELETE
lr
loss
0.025
0.02
0.015
0.01
0.005
0
0.6
0.5
0.4
0.3
0.2
0.1
0
OVERVIEW
TRIALS
DETAILS
YAML
Filter
Enter property name or value
Status
Trial name
Loss
Lr
katib-argo-workflow-245n88r4
0.5497
2.41029e-2
katib-argo-workflow-d77vzfp6
0.4864
1.35818e-2
katib-argo-workflow-hljd4r7n
0.4279
2.33042e-2
katib-argo-workflow-vdb9ddqw
0.421
2.51255e-2

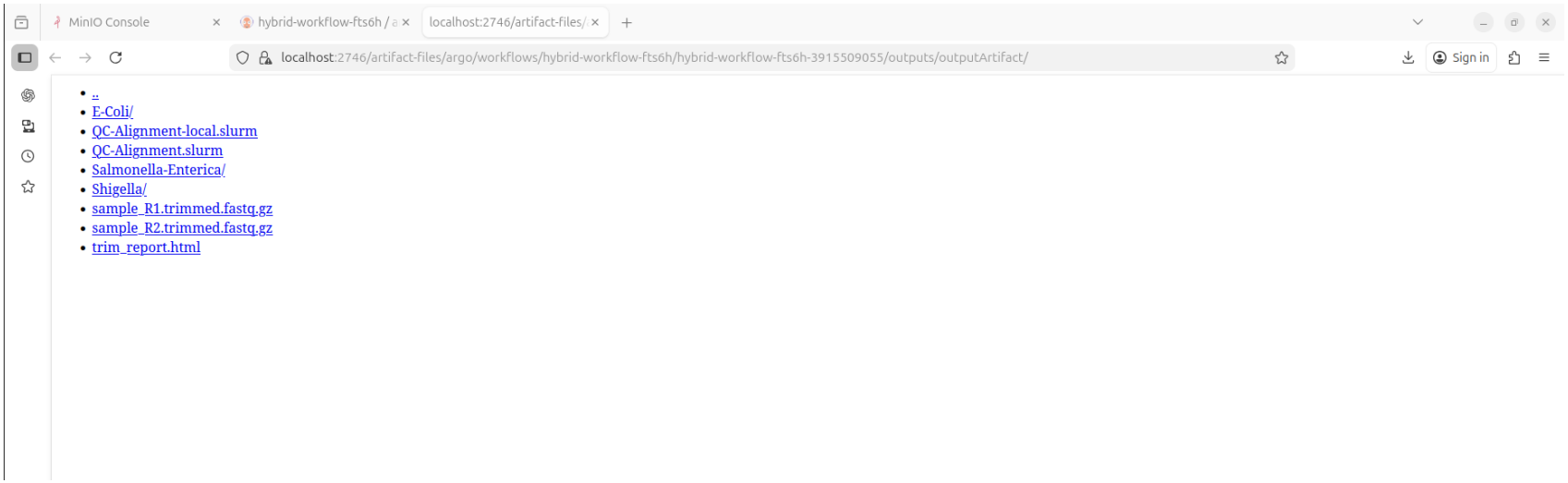
I/O Data on ARIS

```
(base) vsamol@login05:/work2/pa21/vsamol$ ls
total 4
1 ./      1 slurm-job-1186648531/  1 slurm-job-3237751364/
1 ../     1 slurm-job-1281569245/  1 slurm-job-3269050449/
1 conda/  1 slurm-job-1506641641/
(base) vsamol@login05:/work2/pa21/vsamol$ cd slurm-job-1186648531/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-1186648531$ ls
total 261
  1 ./                            1 katib-train.slurm
  1 ../                           1 master_port_8608.txt
  1 data/                         1 mnist_distributed.8608.err
128 distributed-mnist-local.py    1 mnist_distributed.8608.out
128 distributed-mnist.py          1 mnist.log
  1 katib-train-local.slurm
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-1186648531$
```

Genomic QC Alignment Workflow



FastP Trimming Step Output Artifact



QC-Alignment Files on ARIS

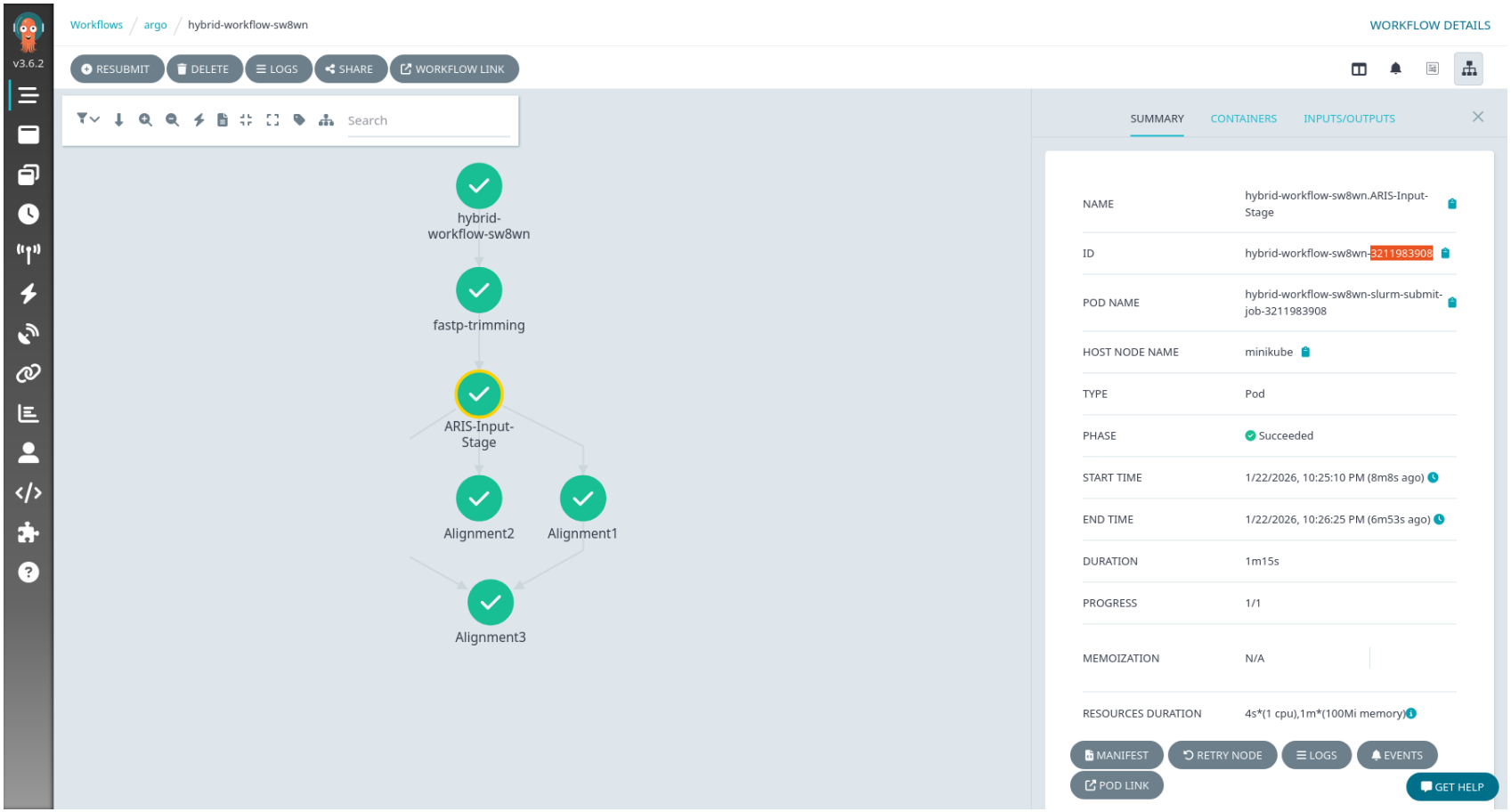
```
vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103
(base) vsamol@login05:/work2/pa21/vsamol$ ls
total 2
1 ./  1 ../  1 conda/  1 slurm-job-2449343103/
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             13635   compute Genomic-   vsamol PD       0:00      1 (AssocGrpJobsLimit)
             13634   compute Genomic-   vsamol  R       0:03      1 m45
             13633   compute Genomic-   vsamol  R       0:04      1 m45
(base) vsamol@login05:/work2/pa21/vsamol$ cd slurm-job-2449343103/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ ls
total 444932
     1 ./        1 E-Coli/                          16384 Genomic-QC-Alignment.13634.out       1 QC-Alignment.slurm    204672 sample_R1.trimmed.fastq.gz       1 Shigella/
     1 ../   16384 Genomic-QC-Alignment.13633.out       1 QC-Alignment-local.slurm             1 Salmonella-Enterica/  206976 sample_R2.trimmed.fastq.gz     512 trim_report.html
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ cd ..
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-2449343103/
4.2G    slurm-job-2449343103/
(base) vsamol@login05:/work2/pa21/vsamol$ cd slurm-job-2449343103/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ ls
total 445060
     1 ./        16384 Genomic-QC-Alignment.13633.out       1 QC-Alignment-local.slurm  204672 sample_R1.trimmed.fastq.gz     512 trim_report.html
     1 ../         128 Genomic-QC-Alignment.13634.out       1 QC-Alignment.slurm        206976 sample_R2.trimmed.fastq.gz
     1 E-Coli/   16384 Genomic-QC-Alignment.13635.out       1 Salmonella-Enterica/           1 Shigella/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ cd Salmonella-Enterica/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103/Salmonella-Enterica$ ls
total 1621506
     1 ./         1 alignment_summary.txt     5760 genome_idx.1.bt2        1 genome_idx.3.bt2     5760 genome_idx.rev.1.bt2  1210880 genome.sam             128 genome.sorted.bam.bai
     1 ../     4992 genome.fa                 1280 genome_idx.2.bt2     1280 genome_idx.4.bt2     1280 genome_idx.rev.2.bt2   390144 genome.sorted.bam
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103/Salmonella-Enterica$ cd ..
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ cd ..
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-2449343103/
5.8G    slurm-job-2449343103/
(base) vsamol@login05:/work2/pa21/vsamol$ cd slurm-job-
-bash: cd: slurm-job-: No such file or directory
(base) vsamol@login05:/work2/pa21/vsamol$ cd slurm-job-2449343103/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ du -sh E-Coli/
1.9G    E-Coli/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ du -sh Salmonella-Enterica/
1.6G    Salmonella-Enterica/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$ du -sh Shigella/
1.9G    Shigella/
(base) vsamol@login05:/work2/pa21/vsamol/slurm-job-2449343103$
```

Alignment Results

```
5363478 + 0 in total (QC-passed reads + QC-failed reads)
0 + 0 secondary
0 + 0 supplementary
0 + 0 duplicates
5361796 + 0 mapped (99.97% : N/A)
5363478 + 0 paired in sequencing
2681739 + 0 read1
2681739 + 0 read2
5346756 + 0 properly paired (99.69% : N/A)
5360888 + 0 with itself and mate mapped
908 + 0 singletons (0.02% : N/A)
0 + 0 with mate mapped to a different chr
0 + 0 with mate mapped to a different chr (mapQ>=5)
```

```
5363478 + 0 in total (QC-passed reads + QC-failed reads)
0 + 0 secondary
0 + 0 supplementary
0 + 0 duplicates
4727968 + 0 mapped (88.15% : N/A)
5363478 + 0 paired in sequencing
2681739 + 0 read1
2681739 + 0 read2
4588950 + 0 properly paired (85.56% : N/A)
4640512 + 0 with itself and mate mapped
87456 + 0 singletons (1.63% : N/A)
2 + 0 with mate mapped to a different chr
2 + 0 with mate mapped to a different chr (mapQ>=5)
```

Custom SchedulerAPI Workflow



ARIS Data Storage clean-up

```
vsamol@login05:/work2/pa21/vsamol
(base) vsamol@login05:/work2/pa21/vsamol$ ls
total 3
1 ./  1 ../  1 conda/  1 slurm-job-2449343103/  1 slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             13636   compute Genomic-   vsamol  R       1:04      1 m45
             13637   compute Genomic-   vsamol  R       1:04      1 m45
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-3211983908/
2.8G    slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             13637   compute Genomic-   vsamol  R       1:53      1 m45
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-3211983908/
2.0G    slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             13637   compute Genomic-   vsamol  R       2:03      1 m45
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-3211983908/
408M    slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             13638   compute Genomic-   vsamol  R       0:05      1 m45
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-3211983908/
472M    slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-3211983908/
520M    slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-3211983908/
1.5G    slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             13638   compute Genomic-   vsamol  R       1:13      1 m45
(base) vsamol@login05:/work2/pa21/vsamol$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-3211983908/
403M    slurm-job-3211983908/
(base) vsamol@login05:/work2/pa21/vsamol$ du -sh slurm-job-2449343103/
5.8G    slurm-job-2449343103/
(base) vsamol@login05:/work2/pa21/vsamol$
```

Results

This thesis addressed the practical challenge of orchestrating end-to-end pipelines that span both cloud-native infrastructure and traditional HPC environments. The core goal of this work was therefore not to replace either system, but to enable them to cooperate within a single workflow model in a way that is deployable on real clusters.

Future Work

- Implementation as a Kubernetes CRD/Operator for better integration with Cloud Native Tools.
- Further Data transfer Optimizations such as predictive algorithms for output size and monitoring of available storage.
- Integration with more Distributed Data Storage Options.
- Extending the SchedulerAPI (Resource Availability).

Thank you!